

ВЕЛИЧЕЗНА ШПОРА ДО КОЛОСА З ООП JAVA

1. БАЗА JAVA

Структура програми

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

- `class` — клас
- `main()` — точка входу
- `System.out.println()` — вивід

2. КЛАСИ ТА ОБ'ЄКТИ

Що таке клас?

Клас — це шаблон для створення об'єктів.

🚩 Klasa — szablon do tworzenia obiektów.

Наприклад:

```
class Car {  
}
```

Тут ми описали "тип" машини.

Що таке об'єкт?

Об'єкт — конкретний екземпляр класу.

🚩 Obiekt — konkretna instancja klasy.

```
Car c = new Car();
```

Коли використовувати?

Коли треба створити структуру з даними та методами.

Наприклад:

- Person
- Car
- Product
- Clock
- Shape

Створення класу

```
public class Person {  
    String name;  
    int age;  
  
    void sayHello() {  
        System.out.println("Hi " + name);  
    }  
}
```

Створення об'єкта

```
Person p = new Person();  
p.name = "Ivan";  
p.age = 20;  
p.sayHello();
```

Конструктор

Конструктор автоматично викликається при створенні об'єкта.

 Konstruktor wywołuje się automatycznie podczas tworzenia obiektu.

Для чого потрібен?

- задати стартові значення
- ініціалізувати поля
- перевірити дані

Коли використовувати?

Майже завжди, коли об'єкт має важливі поля.

Наприклад:

```
Person p = new Person("Ivan");
```

Без конструктора довелось би окремо все присвоювати.

```
public class Person {  
    String name;  
  
    Person(String name) {  
        this.name = name;  
    }  
}
```

this

Посилання на поточний об'єкт.

```
this.name = name;
```

3. ІНКАПСУЛЯЦІЯ

Що це?

Приховування внутрішніх даних класу.

 Enkapsulacja — ukrywanie danych klasy.

Ідея:

НЕ давати доступ напряму до полів.

Тому використовують:

- private
- getter
- setter

Коли використовувати?

ПРАКТИЧНО ЗАВЖДИ для полів класу.

Бо так безпечніше.

private + getter/setter

getter

Повертає значення поля.

🇵🇱 Getter zwraca wartość pola.

setter

Змінює значення поля.

🇵🇱 Setter zmienia wartość pola.

Коли використовувати?

Getter — коли треба прочитати дані.

Setter — коли треба безпечно змінити дані.

```
public class Person {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Навіщо?

- захист даних
- контроль доступу
- безпечна зміна полів

4. НАСЛІДУВАННЯ

Що це?

Один клас отримує можливості іншого.

🇵🇱 Dziedziczenie — jedna klasa przejmuje możliwości drugiej.

Animal -> Dog

Dog успадковує Animal.

Коли використовувати?

Коли є зв'язок типу:

IS-A

Наприклад:

- Dog IS Animal
- Circle IS Shape
- DigitalClock IS Clock

```
class Animal {  
    void sound() {  
        System.out.println("...");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Woof");  
    }  
}
```

Використання

```
Dog d = new Dog();  
d.sound();  
d.bark();
```

5. ПОЛІМОРФІЗМ

Що це?

Один тип може поводитись по-різному.

 Polimorfizm — jeden typ może mieć różne zachowanie.

Приклад:

```
Animal a = new Dog();
```

Коли використовувати?

Коли хочемо працювати з різними об'єктами через спільний тип.

Наприклад:

```
List<Shape>
```

де можуть бути:

- Circle
- Polygon
- Ellipse

```
Animal a = new Dog();  
a.sound();
```

Java викликає метод класу Dog.

Override

```
@Override  
void sound() {  
    System.out.println("Woof");  
}
```

6. АБСТРАКТНІ КЛАСИ

Що це?

Шаблон для інших класів.

 Klasa abstrakcyjna — szablon dla innych klas.

Abstract class зазвичай описує спільну поведінку.

Коли використовувати?

Коли є базовий клас, але його немає сенсу створювати.

Наприклад:

Shape

Бо Shape сам по собі не існує.

Існують:

- Circle
- Polygon
- Ellipse

```
abstract class Shape {  
    abstract double area();  
}  
  
class Circle extends Shape {  
    double r;  
  
    Circle(double r) {  
        this.r = r;  
    }  
  
    double area() {  
        return Math.PI * r * r;  
    }  
}
```

Важливо

- abstract class НЕ можна створити через `new`
- abstract метод не має тіла

7. ІНТЕРФЕЙСИ

Що таке interface?

Контракт методів.

 Interfejs — kontrakt metod.

Interface каже:

"цей клас ОБОВ'ЯЗКОВО має реалізувати ці методи"

Коли використовувати?

Коли різні класи мають однакову поведінку.

Наприклад:

- Drawable
- Serializable
- Comparable

```
interface Drawable {  
    void draw();  
}  
  
class Circle implements Drawable {  
    public void draw() {  
        System.out.println("Drawing");  
    }  
}
```

Різниця abstract vs interface

Abstract	Interface
може мати поля	тільки константи
може мати реалізацію	переважно без реалізації
extends	implements

8. STATIC

Що означає static?

static належить класу, а не об'єкту.

🚩 static należy do klasy.

Коли використовувати?

Коли значення спільне для всіх об'єктів.

Наприклад:

- counter
- utility methods
- Math

static поле

```
class Counter {  
    static int count = 0;  
}
```

static метод

```
static void hello() {  
    System.out.println("Hello");  
}
```

Виклик:

```
Counter.hello();
```

9. FINAL

```
final int x = 5;
```

- final змінна — не змінюється
- final class — не можна успадкувати
- final method — не можна override

10. EXCEPTION

Що таке exception?

Помилка під час виконання програми.

 Exception — błąd podczas działania programu.

Коли використовувати try/catch?

Коли код може впасти.

Наприклад:

- читання файлу
- parse int

- ділення
- робота з мережею

try catch

try

Код який може дати помилку.

🇵🇱 try — kod mogący wygenerować błąd.

catch

Ловить помилку.

🇵🇱 catch przechwytuje błąd.

```
try {  
    int a = 5 / 0;  
}  
catch(Exception e) {  
    System.out.println(e.getMessage());  
}
```

throw

```
throw new Exception("Error");
```

throws

```
void test() throws Exception {  
}
```

11. ВЛАСНІ EXCEPTION

З файлів:

- AmbiguousPersonException
- NegativeLifespanException

Приклад

```
class MyException extends Exception {  
    public MyException(String msg) {  
        super(msg);  
    }  
}
```

Використання:

```
throw new MyException("Bad data");
```

12. КОЛЕКЦІЇ

Що це?

Структури для зберігання багатьох даних.

 Kolekcje — struktury do przechowywania danych.

Коли використовувати?

Коли треба зберігати:

- список
- набір
- таблицю
- пари key/value

ArrayList

Список на основі масиву.

 Lista oparta na tablicy.

Коли використовувати?

Найчастіше.

Коли потрібен:

- список елементів
- швидкий доступ по індексу

```
ArrayList<String> list = new ArrayList<>();  
  
list.add("A");  
list.remove("A");  
list.get(0);
```

LinkedList

```
LinkedList<Integer> list = new LinkedList<>();
```

HashMap

Зберігає пари:

```
key -> value
```

 HashMap przechowuje pary klucz-wartość.

Коли використовувати?

Коли треба швидко знаходити значення по ключу.

Наприклад:

```
"Lublin" -> 100
```

```
HashMap<String, Integer> map = new HashMap<>();  
  
map.put("A", 1);  
map.get("A");
```

HashSet

```
HashSet<Integer> set = new HashSet<>();
```

Не допускає дублікатів.

13. GENERICS (GENERYCZNE)

Що таке generics?

Можливість працювати з різними типами.

🚩 Типу genericzne pozwalają pracować z różnymi typami danych.

Коли використовувати?

Коли клас або метод має працювати з будь-яким типом.

Наприклад:

- List<String>
- List<Integer>
- Box<T>

З папки `generyczne`.

Generic class

```
class Box<T> {  
    T value;  
  
    void set(T value) {  
        this.value = value;  
    }  
  
    T get() {  
        return value;  
    }  
}
```

Використання

```
Box<String> b = new Box<>();  
b.set("Hello");
```

Generic метод

```
public <T> void print(T x) {  
    System.out.println(x);  
}
```

14. FUNCTIONAL INTERFACE

Власний predicate

З файлу CustomPredicate.java

```
interface CustomPredicate<T> {  
    boolean test(T x);  
}
```

Lambda

```
x -> x > 5
```

Приклад

```
CustomPredicate<Integer> p = x -> x > 10;
```

15. STREAM API

Що це?

Інструмент для обробки колекцій.

 Stream API służy do przetwarzania kolekcji.

Коли використовувати?

Коли треба:

- filter
- sort
- map
- count
- reduce

без великих циклів.

filter

Фільтрує елементи.

 filter filtruje elementy.

Коли використовувати?

Коли треба залишити тільки потрібні елементи.

Наприклад:

```
x > 5
```

```
list.stream()  
    .filter(x -> x > 5)  
    .toList();
```

map

Перетворює елементи.

 map przekształca elementy.

Коли використовувати?

Коли треба змінити кожен елемент.

```
list.stream()  
    .map(x -> x * 2)  
    .toList();
```

forEach

```
list.forEach(System.out::println);
```

16. ENUM

```
enum Day {  
    MONDAY,  
    TUESDAY  
}
```

17. FILE I/O

Що це?

Робота з файлами.

🇵🇱 Operacje na plikach.

Коли використовувати?

Коли треба:

- читати CSV
- записувати txt
- парсити дані

Запис

```
FileWriter fw = new FileWriter("a.txt");  
fw.write("Hello");  
fw.close();
```

Читання

```
Scanner sc = new Scanner(new File("a.txt"));
```

18. CSV

Що таке CSV?

Текстовий файл з даними через кому.

🇵🇱 CSV — plik tekstowy z danymi oddzielonymi przecinkami.

Коли використовувати?

Для:

- таблиць
- статистики
- імпорту даних
- колосів 🤖

У проектах використовуються CSV файли.

Приклад рядка

```
name, age  
Ivan, 20
```

split

```
String[] parts = line.split(",");
```

19. UML

З проекту genealogy.

Основні позначення

Символ	Значення
+	public
-	private
#	protected

Наслідування

```
Dog ----|> Animal
```

20. equals() та hashCode()

equals()

Перевіряє чи об'єкти однакові.

 equals() sprawdza równość obiektów.

Коли використовувати?

Коли треба порівнювати об'єкти.

hashCode()

Повертає hash число.

📄 hashCode() zwraca hash obiektu.

Коли використовувати?

Для HashMap та HashSet.

equals

```
@Override
public boolean equals(Object o) {
    return true;
}
```

hashCode

```
@Override
public int hashCode() {
    return Objects.hash(name);
}
```

21. toString()

```
@Override
public String toString() {
    return name;
}
```

22. instanceof

```
if(obj instanceof Dog) {
    System.out.println("Dog");
}
```

23. CASTING

```
Animal a = new Dog();  
Dog d = (Dog)a;
```

24. COMPOSITION

```
class Engine {  
}  
  
class Car {  
    Engine e = new Engine();  
}
```

Car mae Engine.

25. AGGREGATION

```
class Student {  
}  
  
class University {  
    List<Student> students;  
}
```

26. PACKAGE

```
package pl.umcs.oop;
```

import

```
import java.util.ArrayList;
```

27. ACCESS MODIFIERS

Modifier	Доступ
public	всюди
private	тільки в класі
protected	клас + спадкоємці
default	package

28. SUPER

```
super();  
super.name;
```

Виклик батьківського конструктора.

29. ПЕРЕВАНТАЖЕННЯ (OVERLOAD)

Що це?

Методи з однаковою назвою, але різними параметрами.

 Przeciążanie metod.

Коли використовувати?

Коли метод логічно робить те саме, але для різних даних.

```
void print(int x) {}  
void print(String x) {}
```

30. OVERRIDE

Що це?

Перевизначення методу батьківського класу.

 Nadpisanie metody klasy bazowej.

Коли використовувати?

Коли треба змінити поведінку успадкованого методу.

```
@Override  
void print() {}
```

Зміна поведінки батьківського методу.

31. RECORD

```
record Person(String name, int age) {}
```

Автоматично створює:

- constructor
 - getter
 - toString
 - equals
 - hashCode
-

32. OPTIONAL

```
Optional<String> s = Optional.of("abc");
```

Перевірка

```
if(s.isPresent()) {  
}
```

33. LAMBDA

```
(a, b) -> a + b
```

34. METHOD REFERENCE

```
System.out::println
```

35. THREAD

```
Thread t = new Thread();  
t.start();
```

36. SYNCHRONIZED

```
synchronized void test() {  
}
```

37. SERIALIZATION

```
implements Serializable
```

38. MVC

Частина	Роль
Model	дані
View	інтерфейс
Controller	логіка

39. DATABASE CONNECTION

З проекту `site`.

Singleton idea

```
class DatabaseConnection {  
    private static DatabaseConnection instance;  
  
    private DatabaseConnection() {}  
  
    public static DatabaseConnection getInstance() {  
        if(instance == null)  
            instance = new DatabaseConnection();  
  
        return instance;  
    }  
}
```

40. ACCOUNT MANAGER

Ідея з проекту:

- login
- register
- account list
- authentication

Типові методи:

```
login()  
register()  
logout()
```

41. SVG PROJECT

Класи:

- Shape
- Polygon
- Ellipse
- Point
- Segment
- SvgScene

Ідея

ООП для малювання SVG.

Абстрактна фігура

```
abstract class Shape {  
    abstract String toSvg();  
}
```

Наслідування

```
Shape  
├─ Ellipse  
├─ Polygon
```

42. GEOMETRY

Point

```
class Point {  
    double x;  
    double y;  
}
```

Segment

```
class Segment {  
    Point a;  
    Point b;  
}
```

43. CLOCK PROJECT

Класи:

- Clock
- DigitalClock
- City

Наслідування

```
Clock
└─ DigitalClock
```

Можливі питання

- polymorphism
 - override
 - encapsulation
-

44. GENEALOGY PROJECT

Класи:

- Person
- Family

Основна ідея

Родинне дерево.

Зв'язки

```
Family HAS-A Person
```

Exception

- AmbiguousPersonException
 - NegativeLifespanException
-

45. ICD / STATISTICS PROJECT

Класи:

- DeathCauseStatistic
- ICDCodeTabular

Що тут важливе

- читання CSV
- collections

- parsing
 - пошук
 - оптимізація пам'яті
-

46. КЛЮЧОВІ СЛОВА JAVA

Слово	Значення
class	клас
new	створення об'єкта
extends	наслідування
implements	реалізація interface
static	належить класу
final	незмінний
abstract	абстрактний
this	поточний об'єкт
super	батьківський клас
throw	кинути exception
throws	пробросити exception

47. SOLID

S — Single Responsibility

Один клас = одна задача.

O — Open Closed

Відкритий для розширення.

L — Liskov

Спадкоємець має замінити батька.

I — Interface Segregation

Краще багато маленьких interface.

D — Dependency Inversion

Залежність від абстракцій.

48. BIG O

Операція	Складність
Array access	$O(1)$
Loop	$O(n)$
Nested loop	$O(n^2)$
Binary search	$O(\log n)$

49. ЧАСТІ ПИТАННЯ НА КОЛОСІ

Різниця між `abstract class` та `interface`

Abstract:

- може мати state
- constructor
- partial implementation

Interface:

- контракт
 - multiple implements
-

Різниця між `overload` і `override`

Overload

Одна назва, різні параметри.

Override

Перевизначення методу батька.

Різниця між ArrayList і LinkedList

ArrayList	LinkedList
швидкий доступ	швидка вставка
масив	список

HashMap vs HashSet

HashMap:

- key/value

HashSet:

- тільки value
- без повторів

50. МІНІ ШПОРИ

Створення списку

```
List<Integer> list = new ArrayList<>();
```

Цикл foreach

```
for(int x : list) {  
}
```

Scanner

```
Scanner sc = new Scanner(System.in);
```

Random

```
Random r = new Random();
```

Parse int

```
int x = Integer.parseInt("5");
```

String -> char[]

```
char[] arr = s.toCharArray();
```

Довжина

```
s.length();
```

51. ТИПОВІ ЗАДАЧІ

Фільтрація списку

```
list.stream()  
    .filter(x -> x > 10)  
    .forEach(System.out::println);
```

Сортування

```
Collections.sort(list);
```

Comparator

```
list.sort((a,b) -> a-b);
```

52. ЩО ВАРТО ЗАПАМ'ЯТАТИ ПРЯМО ПЕРЕД КОЛОСОМ

1. extends = наслідування
2. implements = interface
3. override = зміна методу
4. overload = інші параметри
5. encapsulation = private + getter/setter

6. polymorphism = Animal a = new Dog()
 7. abstract class не можна new
 8. interface = контракт
 9. static належить класу
 10. try/catch ловить exception
 11. generics = <T>
 12. ArrayList найчастіше використовується
 13. HashSet не має дублікатів
 14. HashMap = key/value
 15. Stream API дуже люблять на колосах
-

53. НАЙКОРОТША ШПОРА В 10 РЯДКІВ

```
extends = наслідування  
implements = interface  
override = перевизначення  
overload = інші параметри  
private = приховування  
polymorphism = один тип, різна поведінка  
abstract = шаблон  
static = спільне для всіх  
try/catch = помилки  
<T> = generics
```

54. МОЖЛИВІ ПИТАННЯ ВІД ВИКЛАДАЧА

Що таке ООП?

Парадигма програмування на основі об'єктів.

4 принципи ООП

1. Інкапсуляція
2. Наслідування
3. Поліморфізм
4. Абстракція

Що таке клас?

Шаблон для створення об'єктів.

Що таке об'єкт?

Екземпляр класу.

Що таке конструктор?

Метод ініціалізації об'єкта.

Що таке interface?

Контракт методів.

Що таке exception?

Помилка під час виконання.

55. МІКРОКОНСПЕКТ ПО ПРОЄКТАМ

genealogy

- family tree
- exceptions
- parsing
- OOP relations

generyczne

- generics
- lambda
- predicate

svg

- abstract shape
- inheritance
- polymorphism
- geometry

site

- singleton
- account manager
- package structure

powtorzenie

- CSV
- statistics
- collections
- optimization

clock

- inheritance
 - digital clock
 - city/time
-

56. SUPER КОРОТКІ ВІДПОВІДІ

Що робить `new`?

Створює об'єкт.

Що робить `static`?

Робить поле/метод спільним.

Що робить `private`?

Приховує поле.

Що робить `extends`?

Успадковує клас.

Що робить `implements`?

Реалізує interface.

Що робить `@Override`?

Позначає override.

Що таке `getter`?

Метод отримання поля.

Що таке `setter`?

Метод зміни поля.

57. ФІНАЛЬНА ШПОРА

ООП:

- encapsulation
- inheritance
- polymorphism
- abstraction

extends -> inheritance
implements -> interface

ArrayList -> список
HashMap -> key/value
HashSet -> без повторів

abstract -> шаблон
interface -> контракт

try/catch -> exception
throw -> кинути
throws -> передати

<T> -> generics
lambda -> x -> x+1
stream -> filter/map

static -> спільне
final -> незмінне

УДАЧІ НА КОЛОСІ 🍀

58. ШПОРИ САМЕ ПО КОЛОСАМ

КОЛОС 2021 — COVID CSV / STATISTICS

Файли:

- confirmed_cases.csv
- deaths.csv

Що треба знати

- читання CSV
- Scanner
- split(",")
- collections
- parsing чисел

- статистика

Типова задача

Прочитати CSV:

```
Scanner sc = new Scanner(new File("data.csv"));

while(sc.hasNextLine()) {
    String line = sc.nextLine();
    String[] parts = line.split(",");
}
```

parse int

```
int x = Integer.parseInt(parts[0]);
```

parse double

```
double x = Double.parseDouble(parts[1]);
```

КОЛОС 2022 — PRODUCT / ZIP

Файли:

- NonFoodProduct.java
- data.zip

Основна тема

ООП + inheritance + products.

Типовий клас

```
class Product {
    String name;
    double price;
}

class NonFoodProduct extends Product {
    int warranty;
}
```

Часті питання

extends

```
class A extends B
```

super

```
super(name, price);
```

override

```
@Override  
public String toString() {  
    return name;  
}
```

КОЛОС 2023 — SVG / MAP PARSER

Файли:

- map.svg
- MapParser.java

Найважливіше

- parsing SVG
- geometry
- XML-like structure
- ООР для shape

Типова abstract структура

```
abstract class Shape {  
    abstract String toSvg();  
}
```

Polygon

```
class Polygon extends Shape {  
}
```

Ellipse

```
class Ellipse extends Shape {  
}
```

Point

```
class Point {  
    double x;  
    double y;  
}
```

Segment

```
class Segment {  
    Point a;  
    Point b;  
}
```

SVG output

```
<circle cx="10" cy="10" r="5" />
```

КОЛОС 2024 — SVG CLOCK / CSV ZONES

Файли:

- zegar.svg
- strefy.csv

Основні теми

- час
- SVG clock
- timezone
- CSV parsing
- collections

LocalTime

```
LocalTime.now();
```

ZonedDateTime

```
ZonedDateTime.now();
```

Читання timezone CSV

```
String[] parts = line.split(",");
```

Clock inheritance

```
class Clock {  
}  
  
class DigitalClock extends Clock {  
}
```

КОЛОС 2025 — MAP / VOIVODESHIP

Файли:

- VoivodeshipMap.java
- kandydaci.txt
- csv files

Теми

- maps
- parsing
- elections/statistics
- collections
- hashmap

HashMap

```
HashMap<String, Integer> map = new HashMap<>();
```

put

```
map.put("Lublin", 100);
```

get

```
map.get("Lublin");
```

foreach map

```
for(String key : map.keySet()) {  
    System.out.println(key);  
}
```

НАЙЧАСТІШІ ЗАДАЧІ З КОЛОСІВ

1. Прочитати CSV

```
Scanner sc = new Scanner(new File("a.csv"));
```

2. Розбити line

```
line.split(",")
```

3. Зберегти в list

```
List<String> list = new ArrayList<>();
```

4. Зберегти в map

```
HashMap<String, Integer> map = new HashMap<>();
```

5. Наслідування

```
class A extends B
```

6. Interface

```
class A implements Drawable
```

7. Override

```
@Override
```

8. Stream filter

```
list.stream().filter(x -> x > 5)
```

9. Sorting

```
Collections.sort(list);
```

10. Exception

```
try {}  
catch(Exception e) {}
```

МЕГА КОРОТКА ШПОРА ПО КОЛОСАМ

```
CSV -> Scanner + split  
SVG -> Shape + Polygon + Point  
Map -> HashMap  
List -> ArrayList  
extends -> inheritance  
implements -> interface  
@Override -> override  
stream/filter -> filtering  
try/catch -> exceptions  
LocalTime -> clock
```

ЩО ВИКЛАДАЧІ ОСОБЛИВО ЛЮБЛЯТЬ

1. parsing CSV

2. abstract Shape
 3. inheritance
 4. polymorphism
 5. collections
 6. HashMap
 7. generics
 8. exceptions
 9. stream API
 10. UML relations
-

СУПЕР ВАЖЛИВІ ШМАТКИ КОДУ

foreach

```
for(String s : list) {  
}
```

lambda

```
x -> x > 5
```

comparator

```
(a,b) -> a-b
```

constructor

```
Person(String name) {  
    this.name = name;  
}
```

getter

```
public String getName() {  
    return name;  
}
```


setter

```
public void setName(String name) {  
    this.name = name;  
}
```

ЩО МОЖЕ БУТИ НА ПРАКТИЧНІЙ ЧАСТИНІ

- прочитати csv
- знайти max/min
- зробити sorting
- створити abstract class
- реалізувати interface
- override toString
- зробити parser
- побудувати SVG
- використати HashMap
- filter через stream

УЛЬТРА-МІНІ ШПОРА

```
CSV = Scanner + split  
SVG = Shape  
Map = HashMap  
List = ArrayList  
extends = inheritance  
implements = interface  
private = encapsulation  
abstract = template  
@Override = polymorphism  
stream/filter = lambda
```